Machine Learning 2

PART 2

Sequence Modelling for Natural Language Processing and Time Series Prediction

Presentación

Profesor: Leonardo Franco

Catedrático de Ciencia de la Computación

Despacho: 3.2.29

e-mail: lfranco@uma.es

Web: http://www.lcc.uma.es/~lfranco/

Horario de tutorías:

Día	Horario	Modalidad	Aula
Lunes	10:30-12:30	Presencial	3.2.29
Lunes	12:30-14:30	Virtual	—
Martes	12:30-14:30	Virtual	—

Horarios | Grado en Ingeniería Informática 2026-27 (plan 2023)

Hora	Lunes	Martes	Miércoles	Jueves	Viernes
Primer Semestre / First term			Aula/Classroom: 3.0.6		
15:30 a 17:15	Aprendizaje Computacional II	ptativas 01.B	Algoritmia y Complejidad	Optativas 01.B	Representación del Conocimiento II
17:30 a 19:15	Robótica	Algoritmia y Complejidad	Representación del Conocimiento II	Robótica	Aprendizaje Computacional II
19:30 a 21:15	Representación del Conocimiento II (franja con restricciones por Doble Grado)	Robótica (franja con restricciones por Doble Grado)	Aprendizaje Computacional I anja con restriccion por Doble Grado)	Algoritmia y Complejidad (franja con restricciones por Doble Grado)	Optativas 01.B

Distribución habitual (mirar el CV por cambios)

Lunes: Grupo Grande - Aula 3.0.6

Miércoles: Grupo reducido 1 – Lab 3.1.10

Viernes: Grupo reducido 2 – Lab 3.1.9

Agenda & Syllabus

Today
P2-Lo: Introduction. Syllabus. Software Setup. Exercise with simple Autoregressive model.

Nov 2-6 :
P2-L1 – Working with Sequences)
Tuesday & Thursdays: reduced groups

Nov 9-13 :
P2-L2 Recurrent Neural Networks

Nov 16-20 :
P2-L3 – LSTM

Nov 23-27:
P2-L4 – Embeddings/Seq2Seq

Dec 1-4 :
P2-L4 – Embeddings/Seq2Seq

Dec 10: Individual projects & revisión

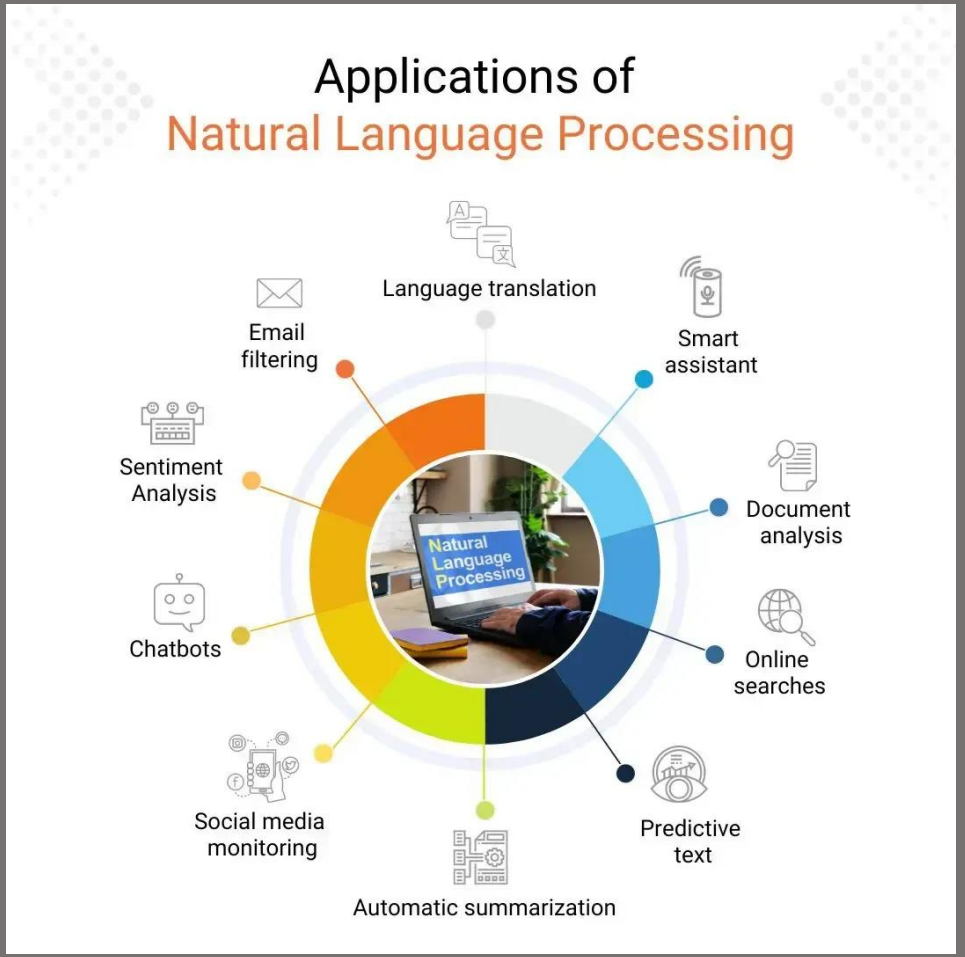
Dec 14-17:
P2-L5 - Transformers

Thu Dec 16th
Evaluation 2nd part

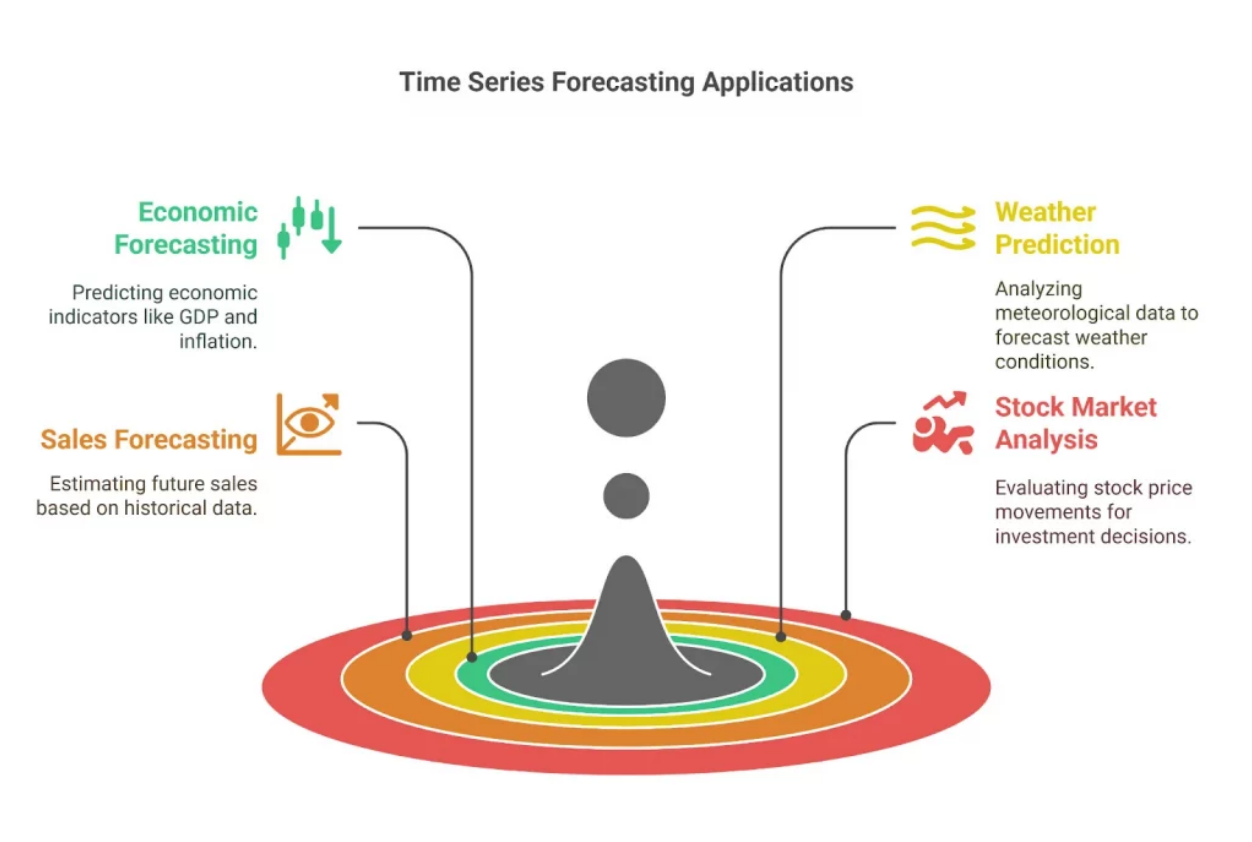
Natural Language Processing (NLP)

the application of computational techniques to the analysis and synthesis of natural language and speech.

It combines computer science, computational linguistics, and machine learning to bridge the gap between human communication and computer comprehension

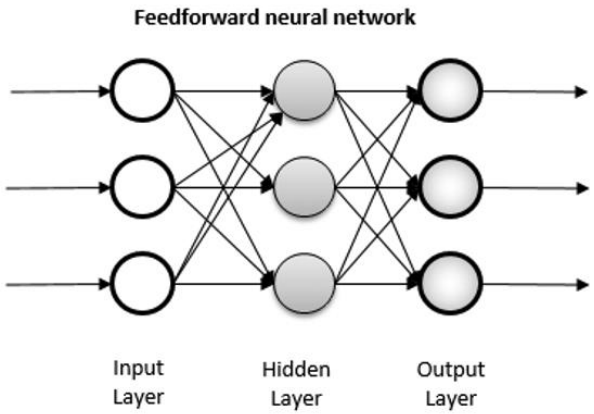


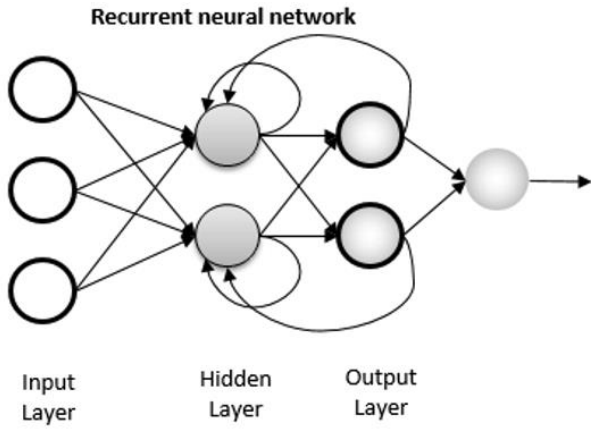
Time series forecasting: process of using historical data to build a model for predicting future values.

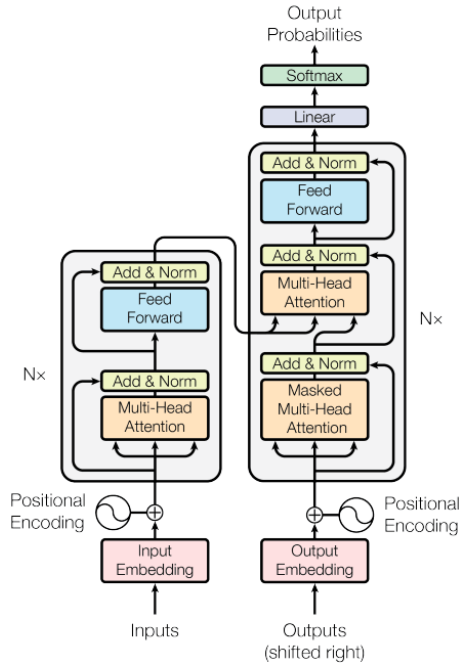


Applications

- Meteorolgy & environment
- Stock market
- Demand forecasting (energy, retail, etc.)
- Medicine (patient monitoring, disease progression,etc.)







From Feed-Forward to Recurrent Neural Networks to Transformer architecture

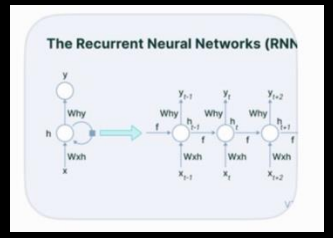
UNIVERSIDAD DE MÁLAGA | uma.es

La evolución de los modelos para análisis de secuencias

1. RNN — Recurrencia (años 1980–2010)

La información se transmite mediante un estado oculto que se actualiza con cada elemento. El orden temporal es explícito, pero el pasado debe comprimirse en un vector.

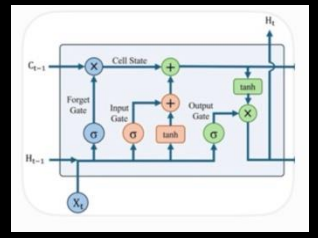
Limitación: gradientes que desaparecen o explotan y dificultades con dependencias largas.



2. LSTM y GRU — Recurrencia con compuertas (1997–2014)

Introducen mecanismos que regulan qué información conservar, actualizar o descartar. Mejoran el aprendizaje de dependencias a largo plazo.

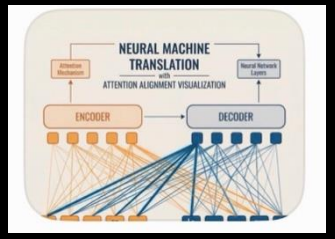
Limitación: el procesamiento sigue siendo secuencial y las dependencias lejanas requieren recorrer muchos pasos.



3. Seq2Seq con atención (2014–2015)

El decodificador deja de depender exclusivamente de un vector resumen: puede consultar diferentes estados del codificador para encontrar la información relevante en cada paso.

Avance: acceso dinámico al contexto de entrada.

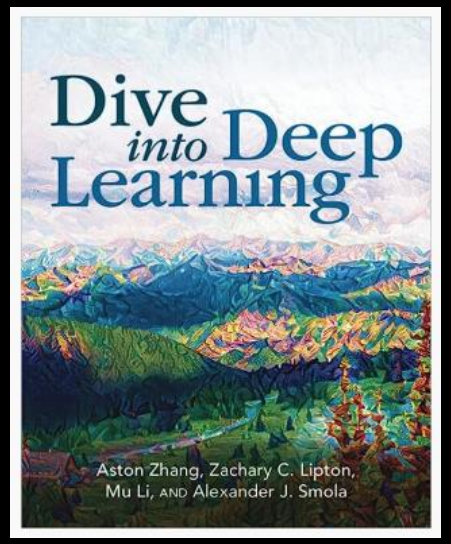


4. Transformer (2017)

Sustituye la recurrencia por bloques de atención y redes feed-forward. Cada posición puede relacionarse directamente con otras posiciones de la secuencia.



Main sources will be class slides, code & papers provided. The following two books are recomended



Dive into Deep Learning

Interactive deep learning book with code, math, and discussions

Implemented with PyTorch, NumPy/MXNet, JAX, and TensorFlow

Adopted at 500 universities from 70 countries



AVAILABLE on-line
https://d2l.ai/index.html

AVAILABLE from Jábega
Malaga University Library

LAB TASKS and EVALUATION

- 5 LABS to follow and learn, and a corresponding exercise to submit, uploading notebooks to the Virtual Campus (upload a notebook file and a pdf describing main experiments and results).

2 Deadlines: 1) 10 days after the lab (complete but initial versión) (15%)

2) Before the exam date (Jan 12th – Final version) (85%)

You cannot have maximum grade if you do not submit a version in the first deadline.

- Second part Test (Wednesday 16/12/2026)

LAB 6: SPECIAL PROJECT

Extension of one the labs to new data.

Application of different techniques.



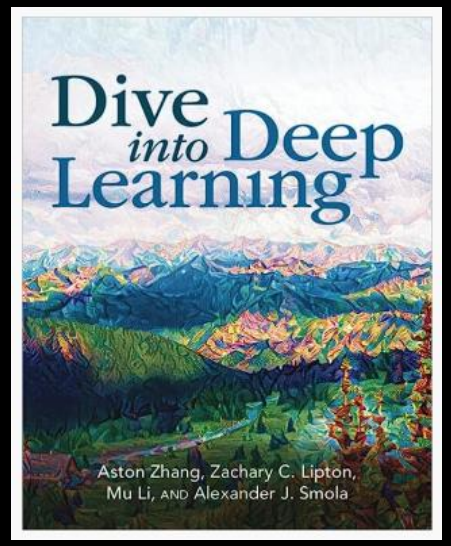
PyTorch is an open source machine learning framework that accelerates the path from research prototyping to production deployment. Built to offer maximum flexibility and speed.

Widely adopted across academia and industry, It supports a broad range of use cases—from natural language processing and computer vision to reinforcement learning and generative AI—through a robust ecosystem of libraries, tools, and integrations.

It is also optimized for performance across CPUs & GPUs.

https://docs.pytorch.org/tutorials/beginner/pytorch_with_examples.html

- PyTorch syntax is designed to be very intuitive and "Pythonic". If you are comfortable with Python libraries like NumPy, working with PyTorch's primary data structure, the **tensor** (an n-dimensional array), will feel very natural.



The Idea is to use Pytorch,
But you can find also notebooks
In Tensorflow.

Dive into Deep Learning

Interactive deep learning book with code, math, and discussions

Implemented with PyTorch, NumPy/MXNet, JAX, and TensorFlow

Adopted at 500 universities from 70 countries

Installing the Deep Learning Framework and the d2l Package

Before installing any deep learning framework, please first check whether or not you have proper GPUs on your machine (the GPUs that power the display on a standard laptop are not relevant for our purposes). For example, if your computer has NVIDIA GPUs and has installed CUDA, then you are all set. If your machine does not house any GPU, there is no need to worry just yet. Your CPU provides more than enough horsepower to get you through the first few chapters. Just remember that you will want to access GPUs before running larger models.

PYTORCH | MXNET | JAX | TENSORFLOW

You can install PyTorch (the specified versions are tested at the time of writing) with either CPU or GPU support as follows:

```
pip install torch==2.0.0 torchvision==0.15.1
```

Our next step is to install the d2l package that we developed in order to encapsulate frequently used functions and classes found throughout this book:

```
!pip install d2l==1.0.0a0
```

LAB 0 : Working with sequences.
Exercise with simple Autoregressive model

We are goint to create some sequential synthetic data and then try to fit it with an autoregressive model.

EX 0: Fit some real data.

Working with Sequences

Image data usually comes in a 2D format. But there are other types of datasets.

Sensor readings that might be available to climate scientists, medical records, stock market data, etc.: data arrives as a collection of sequences, and these samples might be correlated:

think of the next letter in a word, or
the temperature of today in relationship to yesterday.

So, what we are trying to **model are these dependencies** that has quite a lot of information.

For most sequence models, we do not require independence, or even stationarity, we require that the sequences themselves are sampled from some fixed underlying.

Thus the type of data with which we are going to deal is of the form:

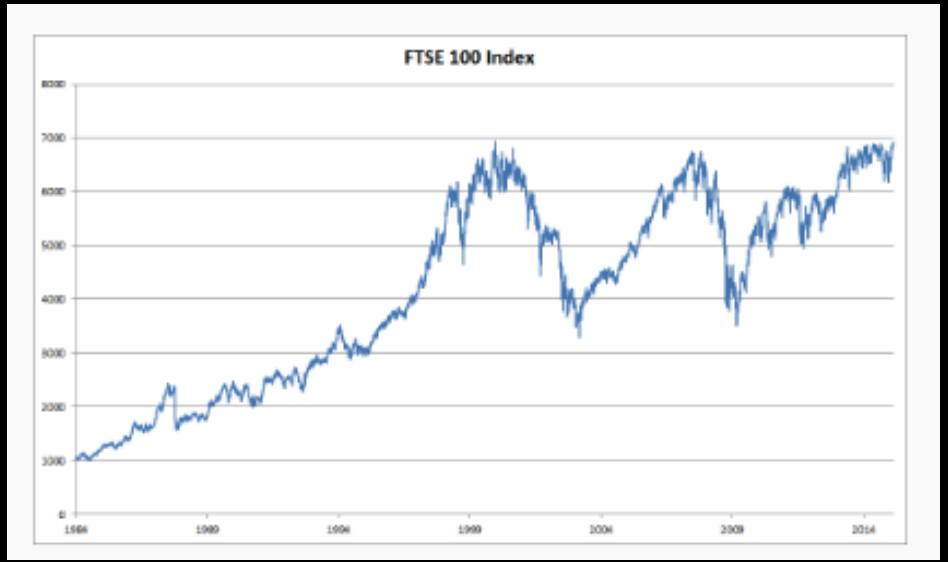
$$[X_1(t=1), X_2(t=1), \ldots, X_n(t=1)] = \underline{X}(1)$$
$$\ldots$$
$$[X_1(t=n), X_2(t=n), \ldots, X_n(t=n)] = \underline{X}(n)$$

Vectors at different times

Simple models: Autoregressive Models

Before introducing specialized neural networks in next classes, let's take a look at some actual sequence data and build up some basic intuitions and statistical tools.

The figure shows stock data from the FTSE 100 index.
At each *time step* ***t*** , we observe the Price **x(t)** of the index at that time.



FTSE 100 index over about 30 years.

A trader might be interested in knowing the probability distribution $P(x_t \mid x_{t-1}, \ldots, x_1)$

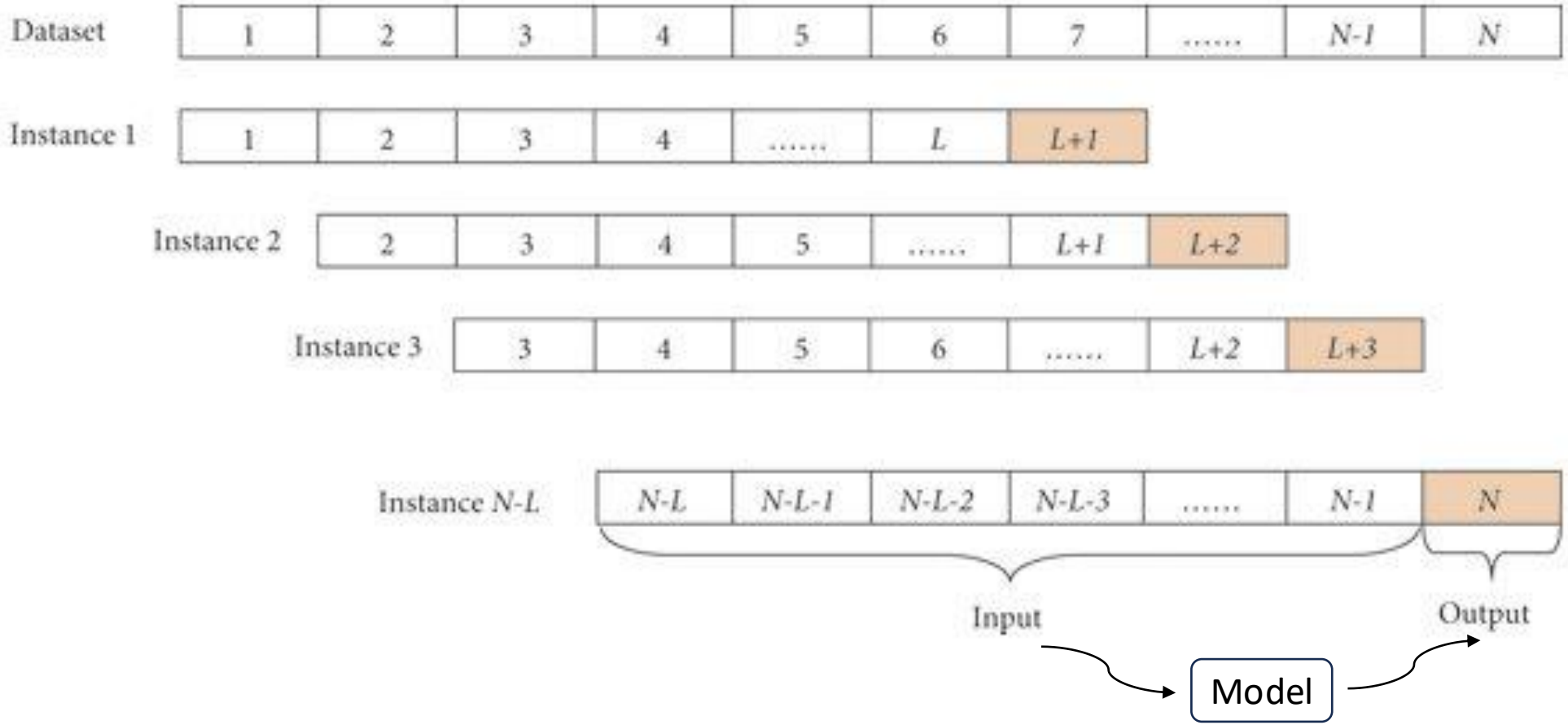
Estimating the **entire distribution** over a continuously valued random variable **can be difficult**, the trader would be happy to focus on a few key statistics of the distribution, particularly the expected value:.

$$\mathbb{E}[(x_t \mid x_{t-1}, \ldots, x_1)],$$

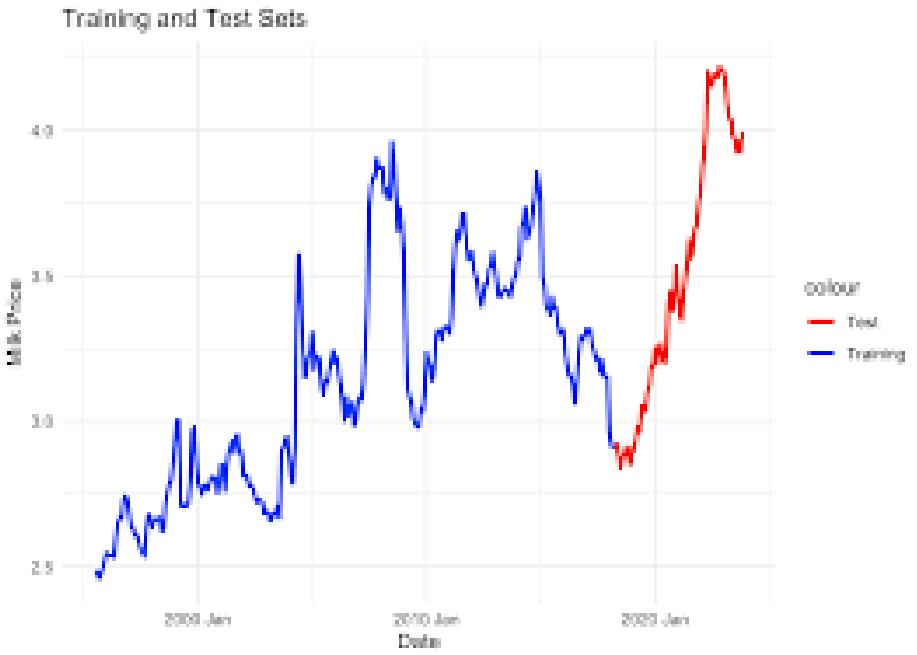
One simple strategy for estimating the conditional expectation would be to apply a **linear regression model. Such models that regress the value of a signal on the previous values of that same signal are naturally called *autoregressive models*.**

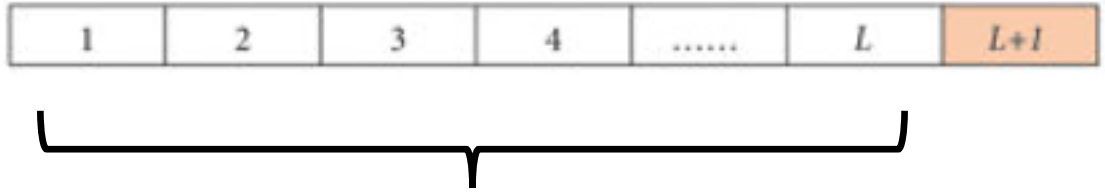
There is just one major problem: the number of inputs, varies, depending on *t* . In other words, the number of inputs increases with the amount of data that we encounter. Thus if we want to treat our historical data as a training set, we are left with the problem that each example has a different number of features

So we are going to define a **window length** and use this number of previous values for estimating the next one.



Supervised training





Input to the trained model

We will use the first 600 time steps for training (learn our model), and then using the same scheme but with no training, we test the next value for time steps from 601 to 1000

$$\min_{c,\phi_1} \sum_{t=2}^{T} (Y_t - (c + \phi_1 Y_{t-1}))^2$$

Linear autoregression

Obtain c and ϕ_1

And then predict future values:

$$\hat{Y}_{t+1} = c + \phi_1 Y_t$$

Markov Models

Now suppose that we wish to employ the strategy mentioned before, where we condition only on the previous τ time steps, i.e., $x_{t-1}, \ldots, x_{t-\tau}$

Whenever we can throw away the history beyond the previous τ steps without any loss in predictive power, we say that the sequence satisfies a *Markov condition*. When τ =1 , we say that the data is characterized by a *first-order Markov model*.

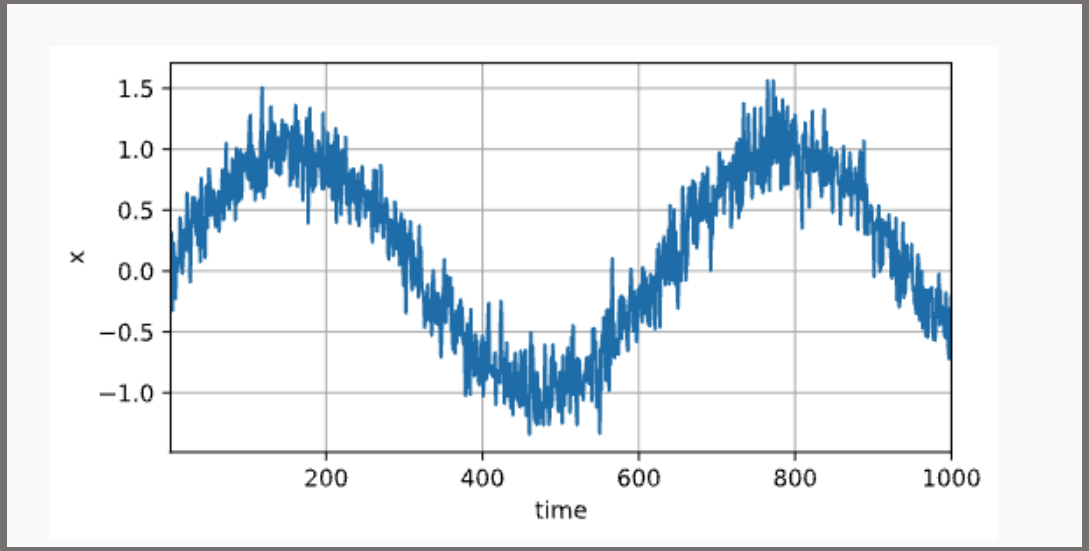
For when the first-order Markov condition holds the factorization of the joint probability becomes a product of probabilities of each word given the previous *word*:

$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1}).$$

Let's generate some sinusoidal data plus noise that we will try to model later

PYTORCH MXNET JAX

```
%matplotlib inline
import torch
from torch import nn
from d2l import torch as d2l
```



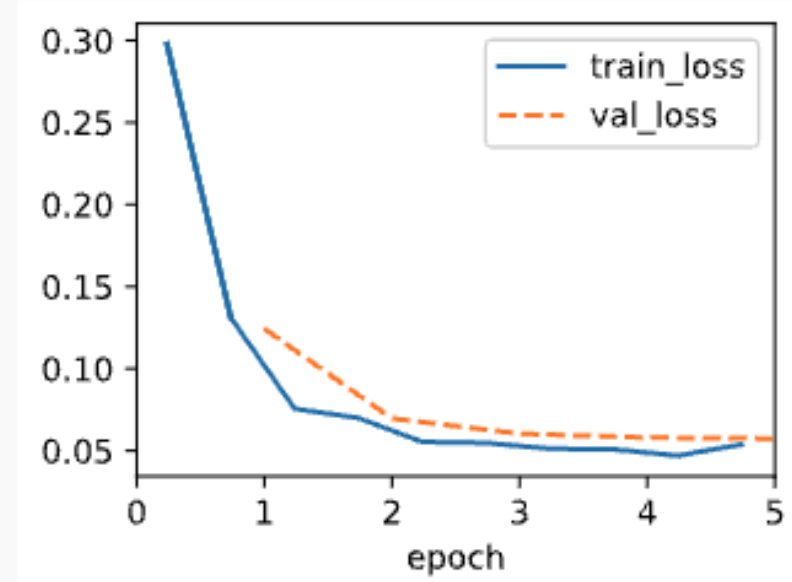

```
class Data(d2l.DataModule):
    def __init__(self, batch_size=16, T=1000, num_train=600, tau=4):
        self.save_hyperparameters()
        self.time = torch.arange(1, T + 1, dtype=torch.float32)
        self.x = torch.sin(0.01 * self.time) + torch.randn(T) * 0.2
data = Data()
d2l.plot(data.time, data.x, 'time', 'x', xlim=[1, 1000], figsize=(6, 3))
```

To begin, we try a model that acts as if the data satisfied a τ -order Markov condition, and thus predicts using only the past τ observations. Thus for each time step we have an example with label $y = x_t$ and features $\mathbf{x}_t = [x_{t-\tau}, \ldots, x_{t-1}]$.

We create a data iterator on the first 600 examples, covering a period of the sin function.

```
@d2l.add_to_class(Data)
def get_dataloader(self, train):
      features = [self.x[i : self.T-self.tau+i] for i in range(self.tau)]
      self.features = torch.stack(features, 1)
      self.labels = self.x[self.tau:].reshape((-1, 1))
            i = slice(0, self.num_train) if train else slice(self.num_train, None)
            return self.get_tensorloader([self.features, self.labels], train, i)
```

```
model = d2l.LinearRegression(lr=0.01)
trainer = d2l.Trainer(max_epochs=5)
trainer.fit(model, data)
```



Training

We setup the data and now:

```
model = d2l.LinearRegression(lr=0.01)
```

specify the model (learning rate, epochs)

```
trainer = d2l.Trainer(max_epochs=5)
```

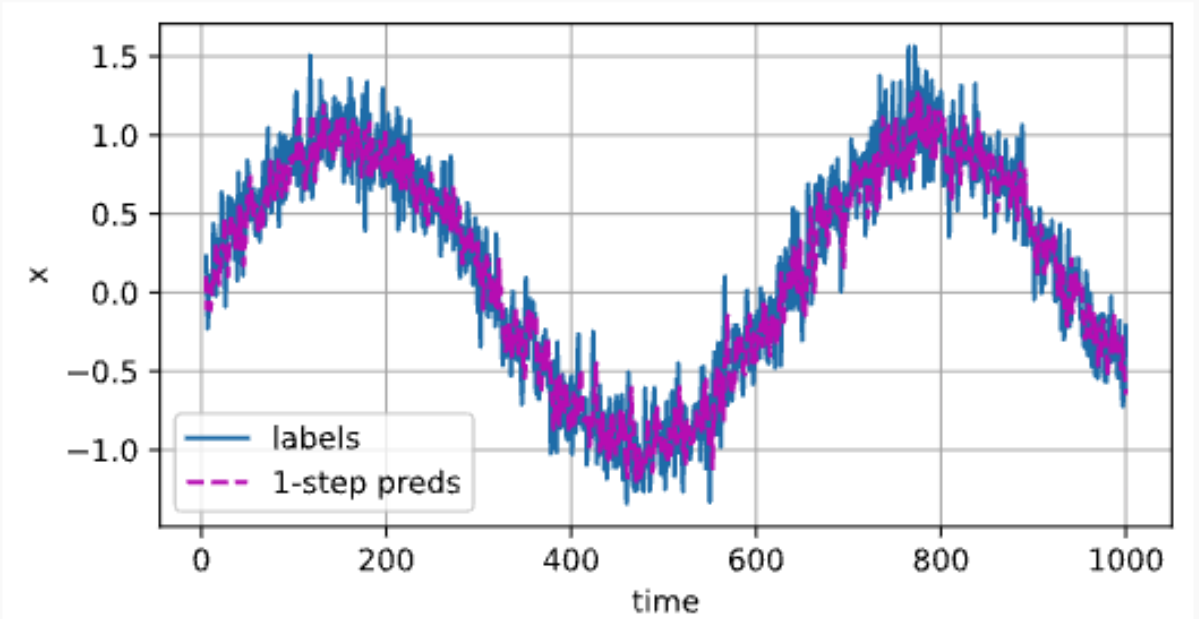
```
trainer.fit(model, data)
```

And Fit the model to the data

To evaluate our model, we first check how well it performs at one-step-ahead prediction.

PYTORCH MXNET JAX TENSORFLOW

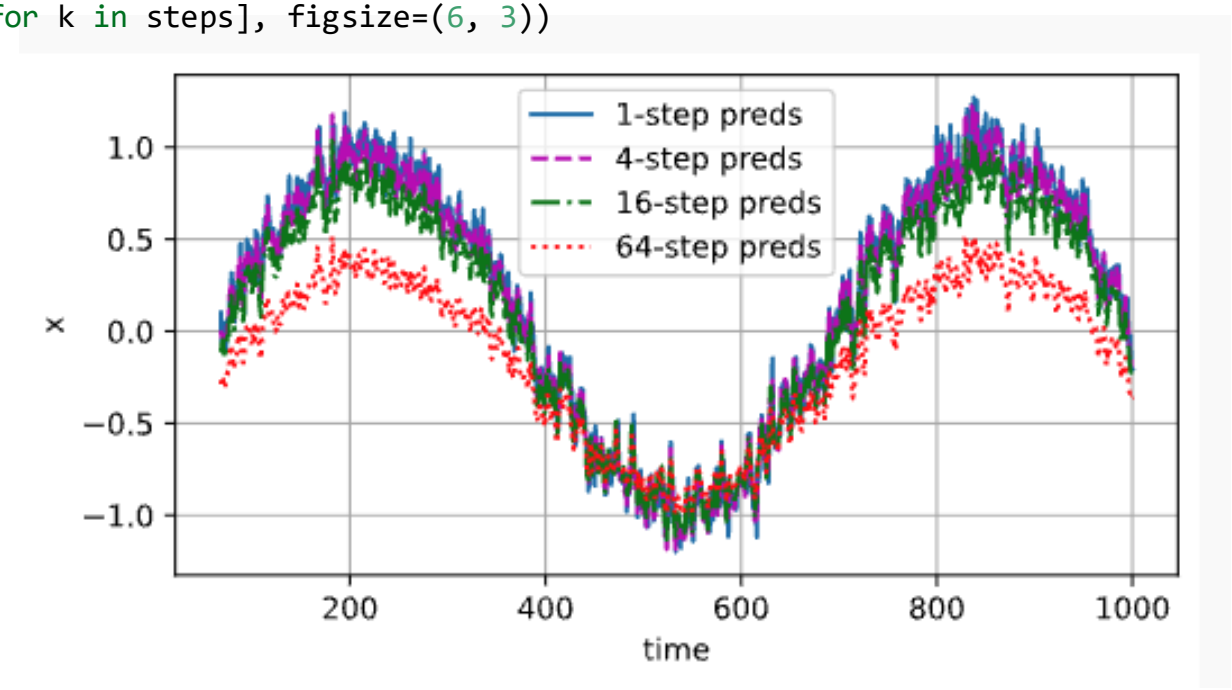
```
onestep_preds = model(data.features).detach().numpy()
d2l.plot(data.time[data.tau:], [data.labels, onestep_preds], 'time', 'x',
         legend=['labels', '1-step preds'], figsize=(6, 3))
```



We just do a visual inspection of the fit, without analyzing now it through a proper metric (like MAE or MSE). We will discuss metrics later.

```
def k_step_pred(k):
features = [] for i in range(data.tau):
    features.append(data.x[i : i+data.T-data.tau-k+1])
# The (i+tau)-th element stores the (i+1)-step-ahead predictions
for i in range(k):
     preds = model(torch.stack(features[i : i+data.tau], 1))
features.append(preds.reshape(-1))
return features[data.tau:]

steps = (1, 4, 16, 64)
preds = k_step_pred(steps[-1])
d2l.plot(data.time[data.tau+steps[-1]-1:], [preds[k - 1].detach().numpy() for k in steps], 'time', 'x',
legend=[f'{k}-step preds' for k in steps], figsize=(6, 3))
```



But we can also do 4, 16 and 64 Ahead Steps prediction

Part 2 Lab 0

- Familiarizarse con el entorno a utilizar. Instalar Pytorch y la librería d2l, ya sea en entorno Jupyter local, o en Colab.

- Abrir y ejecutar el cuaderno provisto. Incorporar diferente número de observaciones anteriores (Tau) en las series y comparar resultados, graficando el error en función de Tau.

- Con un valor elegido de Tau comparar resultados para la predicción de valores futuros a diferentes tiempos, graficando el error.

- Realizar un análisis similar con los datos de valores de bolsa de la empresa IBM (ibm.us.txt) ó con valores de temperatura diarios (daily-min-temperaturas.csv). Extraer conclusiones. Calcular el MAE o MSE en los diferentes casos analizados.

- Teoría: modelos autorregresivos vs. modelos latentes autorregresivos. Busque un ejemplo que requiera variables latentes para capturar la dinámica de los datos. Explique.